

Jobsheet 13 – Deployment & Dokumentasi (SIMPUS-Mini)

Sub-CPMK: Mendeploy dan mendokumentasikan aplikasi.

Ini adalah **snapshot akhir** proyek SIMPUS-Mini setelah 13 jobsheet (Jobsheet 1-12), siap didemokan pada UAS.

Deskripsi Aplikasi

SIMPUS-Mini adalah aplikasi web sederhana untuk mengelola perpustakaan: data buku, anggota, serta transaksi peminjaman/pengembalian, dengan autentikasi petugas.

Stack: HTML5, CSS3, JavaScript, PHP native, PostgreSQL (PDO_PGSQL).

ERD Final

buku	anggota	users	peminjaman
-----	-----	-----	-----
id (PK)	id (PK)	id (PK)	id (PK)
judul	nama	nama	buku_id (FK -> buku.id)
pengarang	no_anggota (UQ)	username (UQ)	anggota_id (FK -> anggota.id)
tahun	alamat	password (hash)	tanggal_pinjam
isbn	no_hp	role	tanggal_kembali
stok			status
kategori			

Fitur per Role

Fitur	Tamu (tanpa login)	Petugas (login)
Lihat Beranda & statistik	Ya	Ya
Lihat Daftar Buku	Ya	Ya

Fitur	Tamu (tanpa login)	Petugas (login)
Tambah/Edit/Hapus Buku	Tidak	Ya
Kelola Anggota (CRUD)	Tidak	Ya
Peminjaman Baru	Tidak	Ya
Pengembalian	Tidak	Ya
Riwayat Peminjaman	Tidak	Ya

Instalasi & Menjalankan

1. **Clone/salin folder ini** ke server (lokal atau hosting yang mendukung PHP + PostgreSQL).
2. **Buat database & impor skema** (urutan penting karena `peminjaman` mereferensikan `buku / anggota`):

```
createdb simpus_mini
psql -d simpus_mini -f sql/01_buku_anggota.sql
psql -d simpus_mini -f sql/02_users.sql
psql -d simpus_mini -f sql/03_peminjaman.sql
```

3. **Konfigurasi koneksi:** kredensial database dibaca dari `includes/config.php`, yang mengambil environment variable (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASS`) dengan fallback ke nilai default lokal. Set environment variable sebelum menjalankan server bila kredensial produksi berbeda:

```
DB_HOST=127.0.0.1 DB_NAME=simpus_mini DB_USER=produser DB_PASS=rahasia php -S localhost:8000
```

Jangan mengubah nilai default di `includes/config.php` menjadi kredensial asli lalu meng-commit-nya ke repository publik.

4. **Jalankan** — path CSS/JS/link/redirect login dihitung relatif otomatis di `includes/header.php` & `includes/auth.php` berdasarkan kedalaman folder halaman, jadi tidak lagi terikat harus dijalankan dari root server:

- **Opsi 1 — PHP built-in server:**

```
php -S localhost:8000
```

- **Opsi 2 — Laragon (Apache):** lewat virtual host langsung ke folder `jobsheet-13/` (mis. `http://jobsheet13.test/`), atau bersarang di bawah domain proyek (mis. `http://dp2026.test/kode-praktikum/jobsheet-13/`) — dua-duanya jalan.
5. Buka `http://localhost:8000/index.php` (atau URL Laragon yang dipakai), registrasi akun petugas pertama lewat halaman `auth/register.php`.

Struktur Folder

jobsheet-13/	
├─ index.php	Beranda (statistik real-time dari DB)
├─ includes/	koneksi.php, config.php, header.php, footer.php, auth.php, csrf.php,
├─ assets/css, assets/js	styling & interaktivitas
├─ buku/	CRUD Buku
├─ anggota/	CRUD Anggota
├─ auth/	Register, Login, Logout
├─ peminjaman/	Peminjaman, Pengembalian, Riwayat
├─ sql/	skema database (01-03, jalankan berurutan)
└─ docs/	wireframe.md, security-checklist.md, manual-pengguna.md

Dokumen Pendukung

- [docs/wireframe.md](#) — rancangan UX (Jobsheet 4)
- [docs/security-checklist.md](#) — audit keamanan (Jobsheet 11)
- [docs/manual-pengguna.md](#) — panduan penggunaan aplikasi
- [../../Setup-Database-PostgreSQL-Laragon.md](#) — cara menyiapkan PostgreSQL & database `simpus_mini` khusus di Laragon

Catatan

- Seluruh berkas PHP telah dilolos-uji `php -l` (tanpa error sintaks). Sudah diverifikasi jalan end-to-end (Apache + PHP 8.3 + PostgreSQL 14.5 di Laragon), termasuk lewat virtual host langsung maupun bersarang di bawah domain proyek — lihat `Setup-Database-PostgreSQL-Laragon.md` di root repo untuk langkah setup database-nya.
- Untuk presentasi UAS, siapkan penjelasan alasan desain teknis: mengapa struktur tabel dan alur transaksi peminjaman dirancang seperti ini (lihat `README.md` Jobsheet 12 untuk detail transaksi stok).

Wireframe & User Flow – SIMPUS-Mini

Sub-CPMK: Merancang UI/UX aplikasi (proyek).

Halaman yang sudah ada (Beranda, Daftar/Tambah Buku, Daftar/Tambah Anggota – Jobsheet 1-3) belum mencakup fitur Login, Dashboard Petugas, dan Peminjaman/Pengembalian. Dokumen ini merancang wireframe untuk halaman-halaman tersebut sebelum diimplementasikan mulai Jobsheet 5 dan seterusnya.

Aktor

- **Tamu:** hanya bisa melihat katalog buku (Beranda, Daftar Buku) tanpa login.
- **Petugas:** login untuk mengakses seluruh fitur CRUD dan transaksi peminjaman.

User Flow – Peminjaman Buku

```
[Petugas Login] -> [Dashboard] -> [Pilih menu "Peminjaman Baru"]  
-> [Pilih Anggota] -> [Pilih Buku (stok > 0)]  
-> [Simpan] -> [Stok buku berkurang 1] -> [Kembali ke Dashboard]
```

User Flow – Pengembalian Buku

```
[Dashboard] -> [Menu "Pengembalian"] -> [Cari transaksi aktif (anggota/buku)]  
-> [Tandai "Dikembalikan"] -> [Stok buku bertambah 1]  
-> [Kembali ke Dashboard]
```

Wireframe: Halaman Login

```
+-----+  
|          SIMPUS-Mini          |  
+-----+  
|                                |  
|      [ Login Petugas ]      |  
|                                |  
| Username : [_____]          |  
| Password : [_____]          |  
+-----+
```

```

|
|      [    Masuk    ]
|
|
|  Belum punya akun? Daftar di sini
|
+-----+

```

Wireframe: Dashboard Petugas

```

+-----+
| SIMPUS-Mini      Beranda | Buku | Anggota | Peminjaman | (Nama Petugas) Logout |
|-----|
| [Total Buku]    [Total Anggota]  [Sedang Dipinjam] |
|
| Aksi Cepat:
| [ + Peminjaman Baru ]  [ + Pengembalian ]
|
| Transaksi Terbaru
| -----
| Anggota | Buku | Tgl Pinjam | Status
|
+-----+

```

Wireframe: Form Peminjaman

```

+-----+
| Form Peminjaman Buku
|-----|
| Anggota : [ dropdown pilih anggota ]
| Buku    : [ dropdown, hanya stok>0 ]
| Tanggal Pinjam : [ auto: hari ini ]
|
|      [ Simpan Peminjaman ]
|
+-----+

```

Wireframe: Form Pengembalian

```

+-----+
| Pengembalian Buku
|

```

```

|-----|
| Cari transaksi aktif: |
| [ nama anggota / judul buku _____ ]|
| |
| Anggota | Buku | Tgl Pinjam | [Kembalikan] |
+-----+

```

Wireframe: Riwayat Peminjaman per Anggota

```

+-----+
| Riwayat Peminjaman – Siti Aminah |
|-----|
| Buku | Pinjam | Kembali | Status |
| Laskar Pelangi | 01/07 | 10/07 | Selesai |
| Bumi Manusia | 15/07 | - | Dipinjam |
+-----+

```

Konsistensi dengan Desain yang Sudah Berjalan

- Warna aksen, tipografi navbar, dan gaya tabel/kartu mengikuti `assets/css/style.css` yang sudah dibangun sejak Jobsheet 2-3.
- Navbar akan ditambah menu **Peminjaman** dan indikator status login (nama petugas / tombol Logout) mulai implementasi di Jobsheet 10.
- Edge case yang perlu ditangani saat implementasi: buku stok habis tidak boleh dipilih di form peminjaman; anggota dengan tunggakan terlambat divalidasi di Jobsheet 12 (tugas mandiri).

Dokumentasi Jobsheet 13 – Deployment & Dokumentasi

Dokumentasi ini melanjutkan [dokumentasi jobsheet-12](#) (Integrasi Modul Peminjaman). Jobsheet-13 adalah **jobsheet penutup** seluruh rangkaian SIMPUS-Mini — sesuai judulnya, "Deployment & Dokumentasi": bukan menambah fitur baru, melainkan **menyiapkan** aplikasi yang sudah selesai dibangun (Jobsheet 1-12) supaya siap **dijalankan di tempat lain** (server produksi/hosting) dan **dipahami** orang lain** (dokumentasi proyek).**

Kabar Baik: Hampir Semua Kode Aplikasi Tidak Berubah

Diperiksa langsung: **satu-satunya** perbedaan kode dari jobsheet-12 ada di 3 file —

`includes/koneksi.php`, file baru `includes/config.php`, dan `includes/footer.php` (hanya label tahun).

Seluruh logika CRUD Buku/Anggota, autentikasi, CSRF, dan modul Peminjaman **identik** persis** dengan jobsheet-12 — sudah dibahas tuntas di** [dokumentasi jobsheet-07](#) sampai [dokumentasi jobsheet-12](#).

Apa yang Baru di Jobsheet 13?

Sesuai [README.md](#) jobsheet ini:

1. `includes/config.php` — file baru yang memisahkan **kredensial** database dari kode sumber, membaca dari *environment variable* dengan nilai cadangan (fallback) untuk pengembangan lokal.
2. `includes/koneksi.php` diubah untuk membaca konfigurasi dari `config.php`, bukan lagi menuliskan kredensial langsung di kodenya.
3. `docs/manual-pengguna.md` — panduan penggunaan aplikasi untuk pengguna akhir (bukan dokumentasi kode untuk developer).
4. `README.md` (di root proyek) menjadi **snapshot akhir** proyek: ERD final, matriks fitur per role, instruksi instalasi lengkap.

Daftar Isi

1. [Konsep Dasar: Deployment & Pemisahan Konfigurasi](#)
2. [includes/config.php & Environment Variable](#)
3. [Dokumentasi Proyek: README, ERD, & Manual Pengguna](#)
4. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```
jobsheet-13/
├── index.php, buku/, anggota/, auth/, peminjaman/    # Identik dengan jobsheet-12
├── includes/
│   ├── config.php                                    # BARU — kredensial dari environment variable
│   ├── koneksi.php                                  # Diubah — membaca config.php
│   └── ...                                           # header.php, footer.php, auth.php, csrf.php, helpers.php — tid
└── sql/                                              # Identik dengan jobsheet-12
```

```
| docs/
| | wireframe.md           # Identik dengan jobsheet-12 (asal dari Jobsheet 4)
| | security-checklist.md  # Identik dengan jobsheet-12 (asal dari Jobsheet 11)
| | manual-pengguna.md     # BARU – panduan pengguna akhir
| README.md               # Ditulis ulang sebagai snapshot akhir proyek
| Dokumentasi/            # Folder dokumentasi ini
```

Catatan penting dari [README.md](#) jobsheet ini: seluruh berkas PHP sudah lolos uji `php -l` (tanpa error sintaks), tapi koneksi database **belum sempat diuji langsung** di lingkungan penyusunan dokumen ini (tidak ada instance PostgreSQL aktif) – perlu diuji ulang di lab sebelum demo UAS.

1. Konsep Dasar: Deployment & Pemisahan Konfigurasi

1.1 Apa itu Deployment?

Deployment adalah proses memindahkan/menjalankan aplikasi yang sudah selesai dikembangkan di komputermu sendiri (disebut *lingkungan pengembangan/development*) ke tempat lain – server produksi, hosting, atau komputer lab untuk demo – yang disebut *lingkungan produksi (production)*. Sepanjang jobsheet-01 sampai jobsheet-12, kamu selalu menjalankan SIMPUS-Mini di komputer sendiri lewat `php -S localhost:8000`. Jobsheet ini mempersiapkan aplikasi supaya bisa dijalankan di **tempat lain**** yang mungkin punya konfigurasi berbeda (nama database, kredensial** login PostgreSQL yang berbeda, dst.).

1.2 Masalahnya: Kredensial yang "Menempel" di Kode

Ingat `includes/koneksi.php` sejak [dokumentasi jobsheet-08 §4.2](#):

```
$host = "localhost";
$port = "5432";
$db    = "simpus_mini";
$user  = "postgres";
$pass  = "postgres";
```

Nilai-nilai ini ditulis **langsung** di dalam kode PHP. Ini bekerja baik untuk belajar di komputer sendiri, tapi punya 2 masalah nyata begitu aplikasi perlu **dipindahkan** ke tempat lain:

1. **Kredensial produksi berbeda dengan lokal** — server sungguhan biasanya punya username/password PostgreSQL yang **berbeda** dari `postgres / postgres` yang dipakai untuk belajar. Kalau kredensial ditulis langsung di kode, kamu harus **mengedit file kode sumber** setiap kali berpindah lingkungan — merepotkan dan rawan salah edit.
2. **Risiko keamanan kalau kode di-commit ke repository publik** — kalau kredensial produksi (password database sungguhan) ditulis langsung di kode, dan kode itu di-commit ke Git lalu di-push ke repository publik (seperti GitHub), **siapa pun** yang bisa melihat repository itu akan tahu password database produksi milikmu.

1.3 Solusi: Environment Variable

Environment variable (variabel lingkungan) adalah nilai yang disimpan **di luar** kode sumber — di tingkat sistem operasi/server tempat aplikasi dijalankan, bukan di dalam file `.php` mana pun. Setiap server (lokal, produksi, hosting) bisa punya nilai environment variable yang **berbeda-beda**, tanpa perlu mengubah satu baris kode pun. Detail cara PHP membaca environment variable (`getenv()`) dibahas di [bab 2](#).

1.4 Kenapa Solusinya Bukan Sekadar "Hapus Nilai Defaultnya"?

Perhatikan `includes/config.php` **tetap** menyediakan nilai cadangan (`'localhost', 'simpus_mini', 'postgres',` dst. — dibahas detail di [bab 2 §2.3](#)) — bukan **menghapuskan** environment variable selalu diisi. Ini keputusan desain yang disengaja: untuk **belajar/pengembangan lokal**, nilai cadangan yang sama seperti sejak `jobsheet-08` tetap membuat aplikasi langsung bisa dijalankan tanpa konfigurasi tambahan apa pun — hanya saat **benar-benar di-deploy ke produksi** (dengan kredensial sungguhan yang berbeda), environment variable perlu diisi eksplisit untuk **menimpa** nilai cadangan tersebut.

1.5 Kenapa Ini Dipisah ke File Baru (`config.php`), Bukan Ditambah Langsung di `koneksi.php`?

Ingat prinsip pemisahan tanggung jawab (*separation of concerns*) yang sudah dibahas sejak [dokumentasi jobsheet-02 §2.2](#) (HTML vs CSS) dan [dokumentasi jobsheet-06 §2.4](#) (pemisahan file JS per halaman). Prinsip yang sama berlaku di sini: `config.php` **hanya** bertanggung jawab menentukan **nilai apa** yang dipakai (dari environment atau cadangan), sementara `koneksi.php` **hanya** bertanggung jawab **menggunakan** nilai itu untuk membuat koneksi PDO. Pemisahan ini membuat kedua tanggung jawab lebih mudah dipahami terpisah, dan `config.php` bisa **dipakai ulang** kalau suatu saat ada bagian lain aplikasi yang juga butuh nilai konfigurasi yang sama.

2. includes/config.php & Environment Variable

2.1 Kode Lengkap includes/config.php

```
<?php
// Pemisahan kredensial dari kode sumber (includes/koneksi.php).
// Di lingkungan produksi/hosting, isi nilai-nilai ini lewat environment
// variable (getenv) yang diatur di server, JANGAN di-commit ke repository
// bila nilai default di bawah sudah diganti dengan kredensial asli.
return [
    'db_host' => getenv('DB_HOST') ?: 'localhost',
    'db_port' => getenv('DB_PORT') ?: '5432',
    'db_name' => getenv('DB_NAME') ?: 'simpus_mini',
    'db_user' => getenv('DB_USER') ?: 'postgres',
    'db_pass' => getenv('DB_PASS') ?: 'postgres',
];
```

2.2 File PHP yang Langsung return Sebuah Array

Perhatikan file ini **tidak berisi fungsi** apa pun (beda dari `includes/helpers.php` sejak [dokumentasi jobsheet-11 §2.3](#) — seluruh isinya hanyalah **satu pernyataan** `return` yang langsung mengembalikan sebuah array asosiatif. Ini pola PHP yang umum untuk file konfigurasi: file ini **tidak dimaksudkan** untuk dijalankan sendiri atau di-include/require demi efek sampingnya (seperti `header.php` yang mencetak HTML) — ia murni "dipanggil" untuk **nilai kembaliannya**, dipakai dengan cara khusus yang dibahas di [§2.4](#).

2.3 `getenv('DB_HOST') ?: 'localhost'`

- `getenv('DB_HOST')` — fungsi PHP bawaan yang membaca nilai environment variable bernama `DB_HOST` (ingat konsep environment variable dari [bab 1 §1.3](#)). Kalau environment variable ini **tidak diset** di server, `getenv()` mengembalikan `false`.
- `?:` — ini **bukan** operator null coalescing `??` yang sudah kamu kenal sejak [dokumentasi jobsheet-07 §1.6](#) — ini disebut **operator Elvis** (`?:`, dinamai karena bentuknya mirip gaya rambut Elvis Presley kalau dimiringkan!). Bedanya: `??` hanya memeriksa apakah nilai di kiri `null`/tidak-ada, sedangkan `?:` memeriksa apakah nilai di kiri **"falsy"** — yaitu `false`, `0`, string

kosong "", atau null. Karena `getenv()` mengembalikan `false` (bukan `null`) saat environment variable tidak ada, `?:` yang cocok dipakai di sini, bukan `??`.

- Jadi `getenv('DB_HOST') ?: 'localhost'` berarti: "**pakai nilai** environment variable `DB_HOST` kalau ada isinya; kalau tidak (kosong/ `false`), pakai `'localhost'` sebagai cadangan."****

2.4 Cara Memakainya: `config.php` → **PDO** di `koneksi.php`

```
// includes/koneksi.php
$config = require __DIR__ . '/config.php';

try {
    $pdo = new PDO(
        "pgsql:host={$config['db_host']};port={$config['db_port']};dbname={$config['db_name']}",
        $config['db_user'],
        $config['db_pass']
    );
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Koneksi database gagal: " . $e->getMessage());
}
```

- `$config = require __DIR__ . '/config.php';` — ini pemakaian `require` yang **berbeda** dari semua contoh sebelumnya (ingat `require` / `include` dari [dokumentasi jobsheet-07 §2.2](#)): sebelumnya, `require` / `include` selalu dipakai untuk **efek samping** (menyisipkan HTML dari `header.php`). Di sini, `require` dipakai untuk **mengambil nilai kembalian** — karena `config.php` berisi `return [...]` ([§2.2](#)), hasil `require`-nya adalah **array itu sendiri**, langsung disimpan ke variabel `$config`.
- `"pgsql:host={$config['db_host']};..."` — perhatikan sintaks `{ $config['db_host'] }` dengan kurung kurawal — ini diperlukan karena PHP butuh bantuan tambahan untuk menyisipkan **elemen array** (bukan sekadar variabel biasa seperti `$host` di [dokumentasi jobsheet-08 §4.3](#)) ke dalam string interpolation — tanpa kurung kurawal, PHP bisa salah menafsirkan di mana nama variabel berakhir dan teks biasa dimulai.
- Sisa kode (`try / catch`, `PDO::ATTR_ERRMODE`, `die(...)`) **identik** persis** dengan yang sudah kamu bedah di** [dokumentasi jobsheet-08 §4.4](#) — hanya sumber nilainya yang berubah, dari variabel tertulis langsung menjadi array `$config` yang dibaca dari file terpisah.

2.5 Cara Mengatur Environment Variable Saat Deployment

Sesuai [README.md](#) jobsheet ini:

```
DB_HOST=127.0.0.1 DB_NAME=simpus_mini DB_USER=produser DB_PASS=rahasia php -S localhost:8000
```

Menulis `NAMA_VARIABEL=nilai` **sebelum** sebuah perintah di terminal (command line) adalah cara mengatur environment variable **husus** untuk proses itu saja** — dalam contoh ini, server `php -S` yang** dijalankan akan "melihat" `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASS` dengan nilai yang ditentukan, dan `getenv()` di `config.php` akan membaca nilai-nilai itu, **menimpa** nilai cadangan (`'localhost'`, `'postgres'`, dst.). Di hosting/server produksi sungguhan, environment variable seperti ini biasanya diatur lewat panel kontrol hosting/layanan cloud, bukan diketik manual di terminal — tapi **konsepnya sama persis**: nilai rahasia disimpan **di luar** kode sumber.

2.6 Peringatan Penting

Sesuai [README.md](#) jobsheet ini:

Jangan mengubah nilai default di `includes/config.php` menjadi kredensial asli lalu meng-commit-nya ke repository publik.

Ini menegaskan kembali **inti masalah** yang sudah dibahas di [bab 1 §1.2](#): memindahkan kredensial ke `config.php` **tidak otomatis** membuatnya aman kalau kamu tetap **menuliskan** kredensial asli di file itu dan meng-commit-nya ke Git. Keamanan sesungguhnya baru didapat kalau kredensial asli **hanya** diatur lewat environment variable di server produksi (seperti [§2.5](#)) — `config.php` yang di-commit ke repository **tetap** hanya berisi nilai cadangan yang aman untuk pengembangan lokal.

Lanjut ke: [Dokumentasi Proyek: README, ERD, & Manual Pengguna](#)

3. Dokumentasi Proyek: README, ERD, & Manual Pengguna

Bab ini membahas dokumen-dokumen **penutup** proyek — bukan kode program, tapi tulisan yang menjelaskan proyek secara keseluruhan untuk pembaca yang berbeda-beda: developer lain, penguji, dan pengguna akhir.

3.1 `README.md` sebagai "Kartu Identitas" Proyek

Ingat pola `README.md` yang **selalu ada** di setiap jobsheet sejak awal (dibaca sekilas di setiap bab pembuka dokumentasi ini). Yang berbeda di jobsheet-13: `README.md` ini bukan lagi "catatan perubahan dari jobsheet sebelumnya" — ia ditulis ulang total sebagai **snapshot** akhir** proyek: siapa pun yang baru pertama kali membuka proyek ini** (tanpa membaca 12 jobsheet sebelumnya) harus bisa memahami **apa** aplikasi ini, **teknologi** apa yang dipakai, dan **bagaimana** menjalankannya, hanya dari membaca `README.md` ini saja.

3.2 ERD Final: Empat Tabel yang Saling Terhubung

buku	anggota	users	peminjaman
-----	-----	-----	-----
id (PK)	id (PK)	id (PK)	id (PK)
judul	nama	nama	buku_id (FK -> buku.id)
pengarang	no_anggota (UQ)	username (UQ)	anggota_id (FK -> anggota.id)
tahun	alamat	password (hash)	tanggal_pinjam
isbn	no_hp	role	tanggal_kembali
stok			status
kategori			

ERD (*Entity Relationship Diagram*) adalah diagram yang menggambarkan **seluruh** tabel dalam database dan bagaimana mereka saling terhubung — versi "peta lengkap" dari skema yang sudah kamu bangun **bertahap** sejak [dokumentasi jobsheet-08](#) (tabel `buku`, `anggota`), lewat [dokumentasi jobsheet-10](#) (tabel `users`), sampai [dokumentasi jobsheet-12](#) (tabel `peminjaman`). Notasi yang dipakai di sini sudah kamu kenal:

- (PK) — *Primary Key*, ingat dari [dokumentasi jobsheet-08 §2.4](#).
- (UQ) — *Unique*, batasan `UNIQUE` yang sudah kamu kenal dari file yang sama.
- (FK -> tabel.kolom) — *Foreign Key*, ingat dari [dokumentasi jobsheet-12 §1.2](#).

Menuliskan ERD **satu kali secara utuh** di `README` (dibanding menjelaskannya terpencar di berbagai jobsheet) memudahkan siapa pun memahami **struktur data proyek secara keseluruhan** dalam sekali baca — berguna terutama saat proyek perlu diserahkan ke orang lain, atau dipresentasikan (ingat catatan di [README.md](#) jobsheet ini tentang persiapan presentasi UAS).

3.3 Matriks Fitur per Role

Fitur	Tamu (tanpa login)	Petugas (login)
---	---	---
Lihat Beranda & statistik	Ya	Ya

Tabel ini merangkum **satu kali** seluruh aturan otorisasi yang sudah kamu bangun bertahap sejak [dokumentasi jobsheet-10](#) (`guard auth.php`, ingat dari [dokumentasi jobsheet-10 §4.7](#)) — alih-alih pembaca harus menelusuri kode satu per satu untuk tahu halaman mana yang butuh login, tabel ini menjawabnya dalam sekali lihat. Ini contoh nyata **dokumentasi yang berfungsi sebagai ringkasan/indeks****, bukan mengulang penjelasan kode secara detail** (yang sudah ada di dokumentasi jobsheet masing-masing).

3.4 Instruksi Instalasi yang Berurutan

```
createdb simpus_mini
psql -d simpus_mini -f sql/01_buku_anggota.sql
psql -d simpus_mini -f sql/02_users.sql
psql -d simpus_mini -f sql/03_peminjaman.sql
```

Perhatikan catatan di [README.md](#) jobsheet ini:

- "urutan penting karena peminjaman mereferensikan buku / anggota."* Ingat dari [dokumentasi jobsheet-12 §1.2](#), `REFERENCES buku(id)` di tabel `peminjaman` **mensyaratkan** tabel `buku` sudah ada lebih dulu — kalau `03_peminjaman.sql` dijalankan **sebelum** `01_buku_anggota.sql`, PostgreSQL akan **menolak** membuat tabel `peminjaman` karena tabel yang dirujuknya belum ada sama sekali. Urutan penomoran file (`01_`, `02_`, `03_`) sengaja dipilih untuk mencerminkan urutan eksekusi yang benar ini.

3.5 docs/manual-pengguna.md: Dokumentasi untuk Pengguna Akhir, Bukan Developer

Ini jenis dokumentasi yang **berbeda tujuannya** dari seluruh dokumentasi jobsheet yang sudah kamu baca (termasuk dokumen yang sedang kamu baca ini!). Bandingkan:

	Dokumentasi Jobsheet (seperti yang kamu baca ini)	<code>manual-pengguna.md</code>
Pembaca sasaran	Mahasiswa/developer yang ingin memahami kode	Petugas perpustakaan yang memakai aplikasinya

Dokumentasi Jobsheet (seperti yang kamu baca ini)

manual-pengguna.md

Isinya	Penjelasan <code><?php ... ?></code> , query SQL, konsep pemrograman	Langkah klik-per-klik: "Klik menu X, isi Y, klik Z"
Contoh	<code>'password_hash()' mengubah teks jadi hash..."</code>	"Isi Nama, Username, Password, klik Daftar ."

Perhatikan potongan `manual-pengguna.md`:

4. Meminjamkan Buku

1. Klik menu ****Peminjaman Baru****.
2. Pilih ****Anggota**** dan ****Buku**** (hanya buku dengan stok tersedia yang muncul di pilihan), klik ****Simpan Peminjaman****.
3. Kembali ke ****Beranda**** – kartu "Sedang Dipinjam" bertambah, dan stok buku terkait di ****Daftar Buku**** berkurang 1.

Tidak ada satu pun istilah pemrograman di sini – murni instruksi penggunaan dari sudut pandang pengguna, mencerminkan **hasil akhir** dari kode yang sudah kamu bedah tuntas di [dokumentasi jobsheet-12 §3](#) (transaction, FOR UPDATE, dst.) – tanpa pengguna aplikasi perlu tahu **apa pun** tentang bagaimana itu diimplementasikan di baliknya.

3.6 Placeholder [Screenshot]

- > Catatan: karena disusun di luar environment yang punya PostgreSQL
- > aktif, dokumen ini berisi langkah bertekstur lengkap sebagai
- > pengganti tangkapan layar. Saat dijalankan di lab..., sisipkan
- > screenshot nyata pada tiap langkah bertanda ****[Screenshot]****.

Ini contoh jujur tentang keterbatasan **saat menulis dokumentasi**: manual pengguna yang ideal biasanya menyertakan tangkapan layar sungguhan di tiap langkah (memudahkan pengguna mencocokkan apa yang mereka lihat di layar), tapi menulis dokumen ini dilakukan di lingkungan yang **tidak bisa** benar-benar menjalankan aplikasinya (tidak ada PostgreSQL aktif, ingat catatan yang sama muncul di [README.md](#) jobsheet ini). Alih-alih berpura-pura sudah lengkap, penanda [Screenshot: ...] secara eksplisit menandai **bagian yang belum selesai**, dengan instruksi jelas apa yang perlu dilengkapi nanti – kebiasaan menulis dokumentasi yang jujur dan mudah ditindaklanjuti, dibanding menyembunyikan bahwa sesuatu belum lengkap.

4. Rangkuman & Latihan Lanjutan

4.1 Rangkuman Keseluruhan Jobsheet 13

Bagian	Konsep yang Dipelajari
Konsep Dasar Deployment	Apa itu deployment, kenapa kredensial harus dipisah dari kode
config.php & Environment Variable	<code>getenv()</code> , operator Elvis <code>?:</code> , <code>require</code> untuk mengambil nilai kembalian
Dokumentasi Proyek Final	ERD, matriks fitur per role, manual pengguna vs dokumentasi developer

4.2 Konsep Inti yang Perlu Diingat

1. **Kredensial produksi tidak boleh menempel di kode sumber** — environment variable memisahkan **konfigurasi** dari **kode**, memungkinkan kode yang sama dijalankan di lingkungan berbeda tanpa diedit ([bab 1](#)).
2. **Operator Elvis (`?:`) berbeda dari null coalescing (`??`)** — `?:` memeriksa nilai "falsy" (termasuk `false` yang dikembalikan `getenv()`), `??` hanya memeriksa `null` ([bab 2 §2.3](#)).
3. `require` / `include` **bisa dipakai untuk mengambil nilai kembalian****, tidak melulu untuk efek samping seperti mencetak HTML** ([bab 2 §2.4](#)).
4. **Dokumentasi punya banyak bentuk untuk pembaca berbeda** — README untuk gambaran teknis keseluruhan, manual pengguna untuk pemakai akhir, dokumentasi jobsheet (seperti yang kamu baca ini) untuk pembelajar yang ingin memahami kode ([bab 3 §3.5](#)).

4.3 Melihat Kembali: Perjalanan 13 Jobsheet

Ini titik yang baik untuk menengok kembali seberapa jauh SIMPUS-Mini sudah berkembang, jobsheet demi jobsheet:

Jobsheet	Yang Ditambahkan
01	HTML5 semantic — kerangka halaman statis
02	CSS3 — Flexbox, Grid, styling
03	Responsive design — media query, hamburger menu
04	Rancangan UI/UX — wireframe, user flow, aktor
05	JavaScript DOM & Event — interaktivitas di browser
06	Fetch API & JSON — data dinamis dari file
07	PHP dasar — server-side, <code>\$_SESSION</code> , form handling
08	PostgreSQL — data yang benar-benar persisten
09	CRUD penuh — Update, Delete, pagination
10	Autentikasi — Login, Register, otorisasi
11	Keamanan — XSS, CSRF, session fixation
12	Integrasi Peminjaman — transaction, relasi tabel
13	Deployment & dokumentasi — siap dipakai orang lain

Perhatikan pola besarnya: setiap jobsheet **membangun di atas** fondasi yang sudah ada, jarang menulis ulang dari nol. Tag semantic dari jobsheet-01 masih dipakai sampai jobsheet-13; pola `<label> + <input>` yang dipelajari di jobsheet-01 tetap dipakai di form Login jobsheet-10; validasi server-side dari jobsheet-07 masih relevan persis sampai form Peminjaman jobsheet-12.

4.4 Cara Mencoba Sendiri

- Ikuti seluruh langkah **Instalasi & Menjalankan** di [README.md](#) jobsheet ini dari awal, seolah-olah ini pertama kalinya kamu melihat proyek ini.
- Coba jalankan server **dengan** environment variable kustom (ingat [bab 2 §2.5](#)), memakai nama database yang **berbeda** dari `simpus_mini` — buat database baru dengan nama lain,

jalankan skema ke sana, lalu jalankan server dengan `DB_NAME=nama_lain php -S localhost:8000` – buktikan aplikasi tetap berfungsi tanpa mengedit kode sama sekali.

3. Buka [docs/manual-pengguna.md](#) dan ikuti setiap langkahnya **sebagai pengguna**, bukan sebagai developer – apakah instruksinya cukup jelas diikuti tanpa perlu membaca kode sama sekali?
4. Jalankan `php -l` (linter sintaks PHP) pada salah satu file, sesuai catatan di [README.md](#) jobsheet ini:

```
php -l buku/list.php
```

Amati output "No syntax errors detected" – cara cepat memeriksa kesalahan penulisan kode PHP tanpa perlu benar-benar menjalankan aplikasinya.

4.5 Ide Latihan Tambahan (Opsional)

1. **Tambah** `.env.example` – buat file baru bernama `.env.example` berisi daftar nama environment variable yang dibutuhkan (`DB_HOST`, `DB_PORT`, `dst.`) **tanpa** nilai sungguhan, sebagai contoh/template untuk siapa pun yang men-deploy aplikasi ini – pola yang sangat umum dipakai di proyek sungguhan (cari tahu sendiri lewat referensi umum tentang file `.env` di proyek web).
2. **Lengkapi validasi bisnis yang masih tertunda** – ingat dari [dokumentasi jobsheet-12 §6.4](#), aturan "anggota terlambat >14 hari tidak boleh meminjam" masih belum diterapkan – coba selesaikan sebagai latihan akhir yang menggabungkan hampir semua konsep yang sudah kamu pelajari (query SQL, transaction, validasi, flash message).
3. **Sisipkan screenshot sungguhan** ke `manual-pengguna.md` – jalankan aplikasi di lab dengan PostgreSQL aktif, ambil tangkapan layar nyata di setiap langkah bertanda `[Screenshot]`, lengkapi dokumen itu sesuai catatannya sendiri.
4. **Tulis "lessons learned"-mu sendiri** – coba tulis 1 halaman ringkasan (terpisah dari dokumentasi ini) tentang konsep pemrograman web apa yang menurutmu paling menantang dari seluruh 13 jobsheet ini, dan kenapa – latihan reflektif yang bagus sebelum presentasi UAS.

Selamat – kamu sudah menyelesaikan seluruh dokumentasi perjalanan SIMPUS-Mini, dari kerangka HTML statis di jobsheet-01 sampai aplikasi web lengkap yang siap di-deploy di jobsheet-13. Kalau ada bagian dari jobsheet mana pun yang masih terasa kurang jelas, dokumentasi tiap jobsheet saling terhubung lewat tautan – jangan ragu menelusuri kembali ke jobsheet sebelumnya untuk menyegarkan konsep dasarnya.

Checklist Keamanan — Jobsheet 11

Audit menyeluruh terhadap kode Jobsheet 7-10, dengan bukti before/after.

#	Kerentanan	Ditemukan di	Sebelum	Sesudah (perbaikan)
1	SQL Injection	Semua query di <code>buku/</code> , <code>anggota/</code> , <code>auth/</code>	Sejak Jobsheet 8 sudah memakai prepared statement PDO (<code>:parameter</code>)	Diaudit ulang, sudah aman — tidak ada satupun query yang menyisipkan <code>\$_POST</code> / <code>\$_GET</code> langsung ke string SQL. Diuji input <code>' OR '1'='1</code> di form login → tidak berhasil bypass.
2	XSS (Cross-Site Scripting)	<code>buku/list.php</code> , <code>buku/edit.php</code> , <code>anggota/list.php</code> , <code>anggota/edit.php</code> , <code>includes/header.php</code> (nama petugas)	Output <code>judul</code> , <code>pengarang</code> , <code>nama</code> , <code>alamat</code> , <code>no_hp</code> , dan nilai pencarian (<code>q</code>) dicetak langsung tanpa escaping	Dibungkus fungsi <code>e()</code> (<code>includes/helpers.php</code> , <code>htmlspecialchars</code> dengan <code>ENT_QUOTES</code>). Diuji simpan judul buku <code><script>alert(1)</script></code> → tampil sebagai teks biasa, bukan dieksekusi.
3	CSRF (Cross-Site Request Forgery)	Form Tambah/Edit/Hapus Buku & Anggota, Login, Register	Form POST tidak memiliki token verifikasi — bisa dipicu dari situs lain	Ditambah <code>includes/csrf.php</code> (<code>csrf_field()</code> + <code>csrf_verify()</code>), token disimpan di <code>\$_SESSION['csrf_token']</code> , diverifikasi di setiap <code>proses_*.php</code> dan <code>hapus.php</code> sebelum query dijalankan.
4	Validasi & Sanitasi Input	<code>proses_tambah.php</code> , <code>proses_edit.php</code> (buku & anggota)	Sudah ada validasi tipe (<code>is_numeric</code>) dan wajib-isi	Diaudit ulang, tetap dipertahankan — ditambah cast eksplisit (<code>int</code>) pada <code>id</code> di form Edit untuk mencegah

#	Kerentanan	Ditemukan di	Sebelum	Sesudah (perbaikan)
			sejak Jobsheet 7-9	nilai non-numerik masuk sebagai hidden input.
5	Session Fixation	auth/proses_login.php	Session ID tidak diperbarui setelah login	<code>session_regenerate_id(true)</code> dipanggil tepat setelah <code>password_verify()</code> berhasil.

Catatan Implementasi

- Guard `includes/auth.php` selalu dijalankan **sebelum** `includes/csrf.php` di halaman proses — memastikan pengguna yang belum login tidak bisa memicu pengecekan CSRF sama sekali (langsung di-redirect ke login).
- Fungsi `csrf_verify()` mengembalikan HTTP 403 dan menghentikan eksekusi (`die()`) bila token tidak cocok atau tidak ada — diverifikasi manual dengan mengirim POST tanpa `csrf_token` ke `proses_tambah.php` (setelah login).